# classifier_notebook

June 8, 2026

## 1 News Headline Classifier: Prompt Engineering Comparison

This notebook implements and compares three prompting strategies for news headline classification using Ollama LLMs.

### 1.1 Import Required Libraries

We start by importing the necessary libraries for building our prompt engineering classifier. These libraries help us: - **os** & **argparse**: Handle file operations and command-line arguments - **re**: Use regular expressions for pattern matching - **ollama**: Interact with local Ollama LLM models - **tabulate**: Format output in nice tables

[1]:
```
import os                       # Provides functions for interacting with the␣
 ↪operating system
import re                       # Provides support for regular expressions␣
 ↪(pattern matching)
import argparse                 # Helps create command-line interfaces and parse␣
 ↪arguments
import ollama                   # Ollama Python API for interacting with local␣
 ↪LLMs
from tabulate import tabulate # Used to print formatted tables in terminal␣
 ↪output
```

#### 1.1.1 Quick Question:

**What's the difference between `import os` and `from tabulate import tabulate`?** - How would you use each one in your code? - Why do you think some imports use `from X import Y` instead of just `import X`?

### 1.2 Define Few-Shot Examples and Categories

This section defines the core data structures used throughout our classifier:

- **FEW_SHOT_EXAMPLES**: 15 labeled headlines across 5 categories that we'll show to the model for few-shot learning. These help the model understand the classification patterns.
- **VALID_CATEGORIES**: The 5 news categories our classifier can predict: Politics, Technology, Sports, Finance, and Health
- **SYNTHETIC_DATASET**: A fallback dataset of 5 headlines with labels, used when real data isn't available

[2]:
```
FEW_SHOT_EXAMPLES = [

    # Politics examples
    {"text": "Senate Approves New Infrastructure Spending Bill.", "label":␣
↪"Politics"},
    {"text": "Prime Minister Announces Snap Election Amid Coalition Collapse.",␣
↪"label": "Politics"},
    {"text": "Governor Signs Executive Order Restricting Industrial Emissions.
↪", "label": "Politics"},

    # Technology examples
    {"text": "New Quantum Computing Chip Breakthrough Multiplies Processing␣
↪Speeds.", "label": "Technology"},
    {"text": "Cybersecurity Breach Exposes Millions of User Accounts Worldwide.
↪", "label": "Technology"},
    {"text": "Startup Launches Revolutionary Virtual Reality Headset for Remote␣
↪Workers.", "label": "Technology"},

    # Sports examples
    {"text": "Underdog Team Secures Dramatic Victory in Championship Final.",␣
↪"label": "Sports"},
    {"text": "Olympic Committee Announces New Host City for Upcoming Summer␣
↪Games.", "label": "Sports"},
    {"text": "World Number One Tennis Star Withdraws From Tournament Due to␣
↪Injury.", "label": "Sports"},

    # Finance examples
    {"text": "Stock Market Plummets as Tech Sector Experiences Massive Sell-Off.
↪", "label": "Finance"},
    {"text": "Global Conglomerate Reports Record Profits in Q3 Financial␣
↪Release.", "label": "Finance"},
    {"text": "Cryptocurrency Regulations Tighten Across European Markets.",␣
↪"label": "Finance"},

    # Health examples
    {"text": "Study Finds Regular Exercise Significantly Reduces Risk of Heart␣
↪Disease.", "label": "Health"},
    {"text": "Hospitals Face Severe Nurse Shortages Amid Seasonal Flu Surge.",␣
↪"label": "Health"},
    {"text": "Researchers Map Human Genome Sequence to Uncover Rare Genetic␣
↪Mutations.", "label": "Health"}
]

VALID_CATEGORIES = [
    "Politics",
    "Technology",
```

```
    "Sports",
    "Finance",
    "Health"
]

SYNTHETIC_DATASET = [

    {
        "headline": "Tech Giants Agree on New Open-Source AI Safety Standards",
        "label": "Technology"
    },

    {
        "headline": "Central Bank Raises Interest Rates by 25 Basis Points to␣
 ↪Combat Inflation",
        "label": "Finance"
    },

    {
        "headline": "Star Striker Signs Record-Breaking Five-Year Contract␣
 ↪Extension",
        "label": "Sports"
    },

    {
        "headline": "Parliament Votes to Pass Historic Climate Action Bill␣
 ↪After Fierce Debate",
        "label": "Politics"
    },

    {
        "headline": "New FDA-Approved Breakthrough Drug Shows Promise in␣
 ↪Halting Alzheimer's",
        "label": "Health"
    },
]
```

#### 1.2.1 Quick Question:

**What's the structure of `FEW_SHOT_EXAMPLES` and why use a list of dictionaries?** - What would happen if you tried to access `FEW_SHOT_EXAMPLES[0]`? - How would you get the label of the first example? (Try: `FEW_SHOT_EXAMPLES[0]["label"]`) - Why do you think we store both `"text"` and `"label"` together instead of separate lists?

### 1.3 Label Cleaning Function

The `clean_label()` function is crucial for extracting valid category labels from raw LLM output.

**Why is this needed?** - Language models don't always output exactly what we ask for - They might include extra text, markdown, or multiple words - This function safely extracts the first valid category found using regex pattern matching

**How it works:** 1. Searches through all valid categories using regex with whole-word matching (`\b`) 2. Returns the first matching category (case-insensitive) 3. Returns "Unknown" if no valid category is found

[3]:
```python
def clean_label(raw_output: str) -> str:

    # Loop through all valid categories
    for category in VALID_CATEGORIES:

        # Use regex search to check if the category exists in output
        # \b ensures whole-word matching
        # re.IGNORECASE ignores capitalization differences
        if re.search(rf"\b{category}\b", raw_output, re.IGNORECASE):

            # Return the matched valid category
            return category

    # If no valid category found, return "Unknown"
    return "Unknown"
```

#### 1.3.1 Quick Question:

**Understanding the `clean_label()` function - what does it really do?** - What's the difference between `re.search(...)` and regular string matching like `if "politics" in raw_output`? - Why use `\b` (word boundaries) instead of just searching for "Politics"? - What happens if the model outputs "My favorite category is Technology" - will it be cleaned correctly? - Try predicting what `clean_label("I think the answer is TECHNOLOGY")` returns

### 1.4 Zero-Shot Classification

**Zero-Shot Prompting** is the simplest approach - the model receives ONLY task instructions with NO examples.

**Key characteristics:** - Minimal prompt tokens (most efficient) - Tests the model's ability to generalize from pre-training - Good baseline to compare other techniques against - Temperature set to 0.0 for deterministic/consistent outputs

**How it works:** 1. Define a system prompt explaining the task and valid categories 2. Send the headline to classify with no examples 3. Extract and clean the model's response 4. Return the prediction along with token counts for efficiency analysis

[4]:
```python
def classify_zero_shot(headline: str, model_name: str) -> tuple[str, int, int]:

    # Create system prompt that instructs the model
    system_prompt = (
```

```
        # Tell the model its role
        "You are an expert news editor. "

        # Explain the task
        "Your task is to classify the provided headline "

        # Show allowed categories
        f"into exactly one of these categories: {VALID_CATEGORIES}. "

        # Strict output formatting instructions
        "Respond with ONLY the category name. "
        "Do not write markdown, intros, or punctuation."
    )

    # Send chat request to Ollama
    response = ollama.chat(

        # Specify which model to use
        model=model_name,

        # Conversation messages
        messages=[

            # System role defines behavior/instructions
            {"role": "system", "content": system_prompt},

            # User message contains the actual headline
            {"role": "user", "content": f"Headline: {headline}"}
        ],

        # Deterministic output
        options={"temperature": 0.0}
    )

    # Extract generated text from response
    raw_content = response['message']['content'].strip()

    # Return:
    # cleaned label,
    # prompt token count,
    # generated token count
    return (
        clean_label(raw_content),
        response.get('prompt_eval_count', 0),
        response.get('eval_count', 0)
    )
```

### 1.4.1 Quick Question:

**What makes Zero-Shot different and why set temperature to 0.0?** - What do you think happens if you change temperature from 0.0 to 1.0? Would answers be identical each time? - What does `tuple[str, int, int]` in the function signature mean? - Why does the function return 3 values instead of just the label? What are `prompt_eval_count` and `eval_count` used for? - The prompt says "Respond with ONLY the category name" - why is this instruction so important to add to the prompt?

## 1.5 Few-Shot Classification

**Few-Shot Prompting** shows the model multiple labeled examples BEFORE asking it to classify a new headline.

**Key characteristics:** - Higher prompt tokens (includes all examples) - Uses in-context learning - the model learns patterns from examples - Often improves accuracy at the cost of more tokens - Still doesn't require fine-tuning the model - Temperature set to 0.0 for deterministic outputs

**How it works:** 1. Create a system prompt explaining the task 2. Format all 15 few-shot examples as: `Headline: [text]` (newline) `Category: [label]` 3. Append the actual headline to classify at the end 4. Send to the model and extract the response 5. Return prediction with token usage metrics

[5]:
```python
def classify_few_shot(
    headline: str,
    examples: list[dict],
    model_name: str
) -> tuple[str, int, int]:

    # System prompt for few-shot prompting
    system_prompt = (

        # Define classification task
        f"Classify the input headline into one of these categories:␣
 ↪{VALID_CATEGORIES}. "

        # Tell model to mimic example format
        "Follow the exact format shown in the examples. "

        # Restrict output format
        "Provide ONLY the category name."
    )

    # Convert each example into formatted text block
    example_blocks = [

        # Example format:
        # Headline: ...
        # Category: ...
```

```
        f"Headline: {ex['text']}\nCategory: {ex['label']}"

        # Iterate through all examples
        for ex in examples
    ]

    # Combine all examples into one user prompt
    # Then append actual query headline
    user_content = (

        # Separate examples using blank lines
        "\n\n".join(example_blocks)

        # Add actual headline to classify
        + f"\n\nHeadline: {headline}\nCategory:"
    )

    # Send request to Ollama
    response = ollama.chat(

        model=model_name,

        messages=[
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": user_content}
        ],

        # Deterministic generation
        options={"temperature": 0.0}
    )

    # Extract model response
    raw_content = response['message']['content'].strip()

    # Return cleaned prediction + token counts
    return (
        clean_label(raw_content),
        response.get('prompt_eval_count', 0),
        response.get('eval_count', 0)
    )
```

### 1.5.1 Quick Question:

**How does Few-Shot use examples and why does it cost more tokens?** - This function takes an `examples` parameter - what if you passed in an empty list `[]` instead? Would it still work? - Look at the line with `"\n\n".join(example_blocks)` - what does this do? Why use `"\n\n"` (double newline) instead of a comma? - If you have 15 examples each with ~10 words, roughly how many more tokens would this use compared to Zero-Shot? - Why do you think providing examples

to the model helps it make better predictions?

### 1.6 Chain-of-Thought Classification

**Chain-of-Thought (CoT) Prompting** encourages the model to reason step-by-step BEFORE giving the final answer.

**Key characteristics:** - Most generated tokens (most reasoning required) - Often improves accuracy on complex tasks - Allows us to see the model's intermediate reasoning - Temperature set to 0.2 for some variety in reasoning - Useful for understanding why a classification was made

**How it works:** 1. Create a system prompt asking for step-by-step reasoning 2. Request output in format: "Final Category: [label]" 3. Extract the step-by-step reasoning and final category 4. Use regex to parse the final category line 5. Return prediction with token counts 6. Fallback to clean_label if the expected format isn't found

[6]:
```
def classify_cot(headline: str, model_name: str) -> tuple[str, int, int]:

    # CoT system prompt
    system_prompt = (

        # Task description
        f"Classify the news headline into one of: {VALID_CATEGORIES}. "

        # Ask the model to reason step-by-step
        "First, reason step-by-step about what fields or industries the words␣
 ↪in the headline relate to. "

        # Force final answer format
        "Finally, end your response with the phrase "
        "'Final Category: <label>'."
    )

    # Query Ollama
    response = ollama.chat(

        model=model_name,

        messages=[
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": f"Headline: {headline}"}
        ],

        # Slightly higher temperature to encourage reasoning
        options={"temperature": 0.2}
    )

    # Extract raw generated output
    raw_content = response['message']['content'].strip()
```

```
    # Use regex to extract:
    # Final Category: <label>
    final_line_match = re.search(
        r"Final Category:\s*(\w+)",
        raw_content,
        re.IGNORECASE
    )

    # If explicit final category exists:
    if final_line_match:

        # Extract label from regex group
        label = clean_label(final_line_match.group(1))

    else:
        # Otherwise try cleaning whole response
        label = clean_label(raw_content)

    # Return prediction + token statistics
    return (
        label,
        response.get('prompt_eval_count', 0),
        response.get('eval_count', 0)
    )
```

#### 1.6.1 Quick Question:

**Understanding Chain-of-Thought and step-by-step reasoning** - Why set temperature to 0.2 here instead of 0.0 like Zero-Shot and Few-Shot? - What does this regex pattern do? `r"Final Category:\s*(\w+)"` - what's the difference between `\s*` and `\w+`? - The function has a try-except pattern: first extract using regex, then fall back to `clean_label()`. Why have this fallback? - If the model writes: "The headline talks about tech, so it's Technology" - will the regex find it or use the fallback? - Why would asking the model to "reason step-by-step" before answering help it classify correctly?

### 1.7 Data Loading Function

The `load_data()` function handles dataset loading with graceful fallback to synthetic data.

**Features:** - Reads headlines from one file (one per line) - Reads corresponding labels from another file (one per line) - Validates that headline and label counts match - Falls back to synthetic dataset if files not found or not specified - Useful for both real datasets and testing/demo scenarios

**Usage:** - With files: `load_data("headlines.txt", "labels.txt")` - With synthetic data: `load_data(None, None)` or `load_data("", "")`

[7]:
```
def load_data(input_file: str, labels_file: str) -> list[dict]:
```

```
    # If both files are provided
    if input_file and labels_file:

        try:

            # Open headline file
            with open(input_file, 'r', encoding='utf-8') as f_in,␣
↪open(labels_file, 'r', encoding='utf-8') as f_lbl:

                    headlines = [
                        line.strip()
                        for line in f_in
                        if line.strip()
                    ]

                    # Read labels
                    labels = [
                        line.strip()
                        for line in f_lbl
                        if line.strip()
                    ]

            # Warn if counts mismatch
            if len(headlines) != len(labels):

                print(
                    f"Warning: Mismatch in lines! "
                    f"{len(headlines)} headlines vs "
                    f"{len(labels)} labels."
                )

            # Combine headlines and labels together
            return [
                {"headline": h, "label": l}
                for h, l in zip(headlines, labels)
            ]

        # Handle missing files
        except FileNotFoundError as e:

            print(f"Error loading files: {e}")

            # Exit program with error
            exit(1)

    else:
```

```
        # Fallback mode
        print(
            "No input files provided via arguments. "
            "Using built-in synthetic dataset."
        )

        return SYNTHETIC_DATASET
```

#### 1.7.1 Quick Question:

**Understanding file handling and data loading** - What does the `with` statement do? Why use it instead of manually calling `.close()` on files? - Explain what `zip(headlines, labels)` does - try this in Python: `list(zip(["A", "B"], [1, 2]))` - What happens if your file has 10 headlines but 8 labels? The code prints a warning - why not just crash? - The line `if line.strip()` filters out empty lines - why is this important? - Why does the function accept `None` as a parameter? What's the advantage of having a synthetic dataset fallback?

### 1.8 Run Evaluation and Compare Strategies

The `run_evaluation()` function is the main orchestrator that:

1. **Tests all three prompting strategies** on every headline in the dataset
2. **Tracks metrics** for each approach:
   - Accuracy: Percentage of correct predictions
   - Prompt tokens: How many tokens the model consumed reading the prompt
   - Generated tokens: How many tokens the model produced
   - Total tokens: Combined token usage (efficiency metric)
3. **Processes each headline through all three methods:**
   - Zero-Shot: Quick but less informed
   - Few-Shot: Balanced approach with examples
   - Chain-of-Thought: Reasoning-based approach
4. **Displays results in a formatted table** showing:
   - Correct predictions per strategy
   - Overall accuracy percentage
   - Average token usage for efficiency comparison

**Output:** Comprehensive comparison showing trade-offs between accuracy and computational cost

[8]:
```
def run_evaluation(dataset: list[dict], model_name: str):

    # Startup information
    print(f"\nInitializing Prompt Engineering Playground...")
    print(
        f"Model: '{model_name}' | "
        f"Total Headlines: {len(dataset)}\n"
    )

    # Dictionary to track metrics for each prompting strategy
    metrics = {
```

```
    "Zero-Shot": {
        "correct": 0,
        "prompt_tok": 0,
        "comp_tok": 0
    },

    "Few-Shot": {
        "correct": 0,
        "prompt_tok": 0,
        "comp_tok": 0
    },

    "Chain-of-Thought": {
        "correct": 0,
        "prompt_tok": 0,
        "comp_tok": 0
    }
}

# Total dataset size
total_items = len(dataset)

# Iterate through dataset
for idx, item in enumerate(dataset, 1):

    # Extract headline
    headline = item["headline"]

    # Extract true label
    ground_truth = item["label"]

    # Progress indicator
    print(f"Processing [{idx}/{total_items}]: \"{headline[:40]}...\"")

    # ------------------------------------------------------------------
    # ZERO-SHOT
    # ------------------------------------------------------------------

    # Run zero-shot classification
    zs_pred, zs_p_tok, zs_c_tok = classify_zero_shot(
        headline,
        model_name
    )

    # Accumulate token statistics
    metrics["Zero-Shot"]["prompt_tok"] += zs_p_tok
```

```
        metrics["Zero-Shot"]["comp_tok"] += zs_c_tok

        # Update accuracy count
        if zs_pred == ground_truth:
            metrics["Zero-Shot"]["correct"] += 1

        # ------------------------------------------------------------------
        # FEW-SHOT
        # ------------------------------------------------------------------

        fs_pred, fs_p_tok, fs_c_tok = classify_few_shot(
            headline,
            FEW_SHOT_EXAMPLES,
            model_name
        )

        metrics["Few-Shot"]["prompt_tok"] += fs_p_tok
        metrics["Few-Shot"]["comp_tok"] += fs_c_tok

        if fs_pred == ground_truth:
            metrics["Few-Shot"]["correct"] += 1

        # ------------------------------------------------------------------
        # CHAIN-OF-THOUGHT
        # ------------------------------------------------------------------

        cot_pred, cot_p_tok, cot_c_tok = classify_cot(
            headline,
            model_name
        )

        metrics["Chain-of-Thought"]["prompt_tok"] += cot_p_tok
        metrics["Chain-of-Thought"]["comp_tok"] += cot_c_tok

        if cot_pred == ground_truth:
            metrics["Chain-of-Thought"]["correct"] += 1

    # ------------------------------------------------------------------
    # PRINT RESULTS
    # ------------------------------------------------------------------

    print("\n" + "=" * 70)

    print("                  PROMPT DESIGN EVALUATION RESULTS                ")

    print("=" * 70 + "\n")
```

```
    # Data structure for tabulate
    table_data = []

    # Process metrics for each approach
    for approach, data in metrics.items():

        # Accuracy percentage
        accuracy = (data["correct"] / total_items) * 100

        # Average prompt tokens
        avg_prompt = data["prompt_tok"] / total_items

        # Average generated tokens
        avg_comp = data["comp_tok"] / total_items

        # Total average tokens
        avg_total = avg_prompt + avg_comp

        # Add row to table
        table_data.append([
            approach,
            f"{data['correct']}/{total_items}",
            f"{accuracy:.1f}%",
            f"{avg_prompt:.1f}",
            f"{avg_comp:.1f}",
            f"{avg_total:.1f}"
        ])

    # Table headers
    headers = [
        "Approach",
        "Correct",
        "Accuracy",
        "Avg Prompt Tok",
        "Avg Gen Tok",
        "Avg Total Tok"
    ]

    # Print formatted table
    print(tabulate(
        table_data,
        headers=headers,
        tablefmt="grid"
    ))
```

### 1.8.1 Quick Question:

**Understanding metrics calculation and loops** - Look at the `metrics` dictionary - why organize results by approach (Zero-Shot, Few-Shot, etc)? - What does `enumerate(dataset, 1)` do differently from just `for item in dataset`? - The accuracy calculation is `(data["correct"] / total_items) * 100` - what would happen if you forgot the `* 100`? - Why are token metrics accumulated (`+=`) while accuracy is just counted (`+= 1` if correct)? - How would you modify this function to only run Zero-Shot classification? What single line would you need to remove? - What's the purpose of printing the progress `[idx/{total_items}]` during execution?

## 1.9 Example Usage

This final section demonstrates how to use the classifier:

**What it does:** 1. Loads the synthetic dataset (5 sample headlines with labels) 2. Runs the full evaluation with the `llama3` model 3. Produces a detailed comparison of all three prompting strategies

**To use with your own data:**

```
# Load your custom dataset
dataset = load_data("your_headlines.txt", "your_labels.txt")

# Run evaluation
run_evaluation(dataset, "llama3")
```

**Requirements:** - Ollama must be installed and running locally - The specified model (default: llama3) must be pulled in Ollama - The model should be able to follow instructions well for best results

**Expected Output:** A comparison table showing accuracy and token efficiency for each prompting strategy

[9]:
```
# Example usage: Run evaluation with synthetic dataset
if __name__ == "__main__":
    # Load dataset (uses synthetic dataset by default)
    dataset = load_data("headlines.txt", "labels.txt")

    # Run evaluation with llama3 model
    run_evaluation(dataset, "llama3")
```

```
Initializing Prompt Engineering Playground…
Model: 'llama3' | Total Headlines: 50

Processing [1/50]: "Senate Passes Landmark Voting Rights Leg…"
Processing [2/50]: "Mayor Announces New City-Wide Public Tra…"
Processing [3/50]: "Diplomatic Tensions Rise Over Disputed B…"
Processing [4/50]: "Gubernatorial Candidate Outlines Tax Ref…"
Processing [5/50]: "Supreme Court Hears Arguments on Controv…"
Processing [6/50]: "Prime Minister Faces Vote of No Confiden…"
```

```
Processing [7/50]: "City Council Approves Zoning Changes for…"
Processing [8/50]: "President Signs Executive Order on Immig…"
Processing [9/50]: "Local Elections See Record Voter Turnout…"
Processing [10/50]: "Parliament Debates Healthcare Budget Cut…"
Processing [11/50]: "Next-Generation Quantum Computer Achieve…"
Processing [12/50]: "Major Smartphone Brand Unveils Foldable …"
Processing [13/50]: "Cybersecurity Firm Detects Widespread Ra…"
Processing [14/50]: "AI Startup Raises $100 Million in Series…"
Processing [15/50]: "Social Media Platform Introduces New Pri…"
Processing [16/50]: "Tech Giant Faces Antitrust Probe Over Ap…"
Processing [17/50]: "Self-Driving Car Completes First Cross-C…"
Processing [18/50]: "Semiconductor Shortage Expected to Persi…"
Processing [19/50]: "New Open-Source Language Model Challenge…"
Processing [20/50]: "Space Tourism Company Successfully Launc…"
Processing [21/50]: "Underdog Tennis Player Wins Grand Slam i…"
Processing [22/50]: "Star Quarterback Traded to Rival Team in…"
Processing [23/50]: "National Soccer Team Secures Spot in Wor…"
Processing [24/50]: "Olympic Gymnast Announces Retirement Aft…"
Processing [25/50]: "Local High School Basketball Team Wins S…"
Processing [26/50]: "Heavyweight Champion Defends Title with …"
Processing [27/50]: "Golf Legend Returns to The Masters After…"
Processing [28/50]: "Marathon Runner Breaks World Record on H…"
Processing [29/50]: "College Football Conference Announces Ma…"
Processing [30/50]: "Rookie Pitcher Throws Perfect Game in Ma…"
Processing [31/50]: "Stock Market Rallies as Inflation Shows …"
Processing [32/50]: "Central Bank Surprises Markets with Aggr…"
Processing [33/50]: "Cryptocurrency Exchange Files for Bankru…"
Processing [34/50]: "Global Tech Conglomerate Reports Record …"
Processing [35/50]: "Oil Prices Surge Following Production Cu…"
Processing [36/50]: "Retail Giant Acquires E-Commerce Competi…"
Processing [37/50]: "Investors Flock to Safe Haven Assets Ami…"
Processing [38/50]: "Housing Market Cools Down as Mortgage Ra…"
Processing [39/50]: "Venture Capital Funding in Biotech Start…"
Processing [40/50]: "National Debt Reaches New Milestone Prom…"
Processing [41/50]: "FDA Approves Revolutionary Gene Therapy …"
Processing [42/50]: "New Study Links Processed Foods to Highe…"
Processing [43/50]: "Hospitals Prepare for Expected Surge in …"
Processing [44/50]: "Researchers Discover Potential Biomarker…"
Processing [45/50]: "Global Health Organization Declares End …"
Processing [46/50]: "Breakthrough in mRNA Technology Could Le…"
Processing [47/50]: "Local Clinics Report Shortages of Essent…"
Processing [48/50]: "Mental Health Awareness Campaign Focuses…"
Processing [49/50]: "Dietary Supplements Face Stricter Regula…"
Processing [50/50]: "Wearable Fitness Trackers Shown to Impro…"

======================================================================
                  PROMPT DESIGN EVALUATION RESULTS
======================================================================
```

```
+------------------+-----------+------------+------------------+--------------
+-----------------+
| Approach         | Correct   | Accuracy   |   Avg Prompt Tok |   Avg Gen Tok |
Avg Total Tok |
+==================+===========+============+==================+==============+
=================+
| Zero-Shot        | 46/50     | 92.0%      |             82.9 |           2   |
84.9 |
+------------------+-----------+------------+------------------+--------------
+-----------------+
| Few-Shot         | 48/50     | 96.0%      |            358.9 |           2   |
360.9 |
+------------------+-----------+------------+------------------+--------------
+-----------------+
| Chain-of-Thought | 45/50     | 90.0%      |             86.9 |         128.2 |
215.1 |
+------------------+-----------+------------+------------------+--------------
+-----------------+
```

#### 1.9.1 Quick Question:

**Understanding when code runs and how to use it** - What does `if __name__ == "__main__":` do? Why is this block NOT executed when running this code in Jupyter? - How would you modify this code to load your own files instead of the synthetic dataset? - What would you change to test with a different model, like "llama2" or "neural-chat"? - In Jupyter, how do you actually run the evaluation? Do you need the `if __name__` block or can you just call `run_evaluation()` directly? - Why would you want to run this evaluation - what insights does comparing the three strategies give you?

### 1.10 Knowledge Check: Prompting Strategies

Test your understanding of the three prompting strategies implemented in this notebook.

#### 1.10.1 Question 1: Zero-Shot vs Few-Shot

**Why might Few-Shot prompting use significantly more prompt tokens than Zero-Shot?** - A) It requires larger LLM models - B) It includes multiple example headlines and labels to help the model learn patterns - C) It uses higher temperature settings - D) It requires special hardware

#### 1.10.2 Question 2: Temperature Parameter

**What is the purpose of setting temperature to 0.0 in Zero-Shot and Few-Shot classification?** - A) To make the model run faster - B) To reduce computational costs - C) To ensure deterministic/consistent outputs without randomness - D) To limit the number of tokens generated

#### 1.10.3 Question 3: Regex Pattern Matching

**In the `clean_label()` function, what does `\b` ensure in the regex pattern?** - A) Beginning of string - B) Whole-word matching (word boundaries) - C) Backspace character - D) Optional

whitespace

#### 1.10.4 Question 4: Chain-of-Thought Reasoning

**Why does Chain-of-Thought typically produce more generated tokens than Zero-Shot?** - A) It uses more examples - B) It asks for step-by-step reasoning before the final answer - C) It requires pre-training on larger datasets - D) It has stricter output formatting requirements

#### 1.10.5 Question 5: Code Understanding

**What does the line `for h, l in zip(headlines, labels)` do in the `load_data()` function?** - A) Concatenates headlines and labels - B) Pairs each headline with its corresponding label at the same index - C) Filters matching headlines and labels - D) Sorts headlines alphabetically